

Attributes reflection

Document #: P3385R1
Date: 2024-10-15
Project: Programming Language C++
Audience: sg7
Reply-to: Aurelien Cassagnes
<aurelien.cassagnes@gmail.com>
Roman Khoroshikh
<rkhoroshikh@bloomberg.net>
Anders Johansson
<ajohansson12@bloomberg.net>

Contents

- 1 Revision history
- 2 Introduction
 - 2.1 Earlier work
 - 2.2 Scope
 - 2.3 Optionality rule
- 3 Proposed Features
 - 3.1 `std::meta::info`
 - 3.2 Reflection operator
 - 3.3 Splicers
 - 3.4 Metafunctions
 - 3.5 Queries
- 4 Proposed wording
 - 4.1 Language
 - 4.2 Library
- 5 Future direction
 - 5.1 Scoped attributes
 - 5.2 Implementation
- 6 Conclusion
- 7 References

§ 1 Revision history

Since [P3385R0]

- Rebase on [P2996R5]
- Reword ignorability section to mention set of rules
- Discuss argument clause in `std::meta::info` equal operator
- Discuss appertaining to null statement in reflect expression

§ 2 Introduction

Attributes are used to a great extent, and there likely will be new attributes added as the language evolves.

As reflection makes its way into our standard, what is missing is a way for generic code to look into the attributes appertaining to an entity. That is what this proposal aims to tackle.

A motivating example is the following

```
enum class Result {
    success,
    warn,
    fail,
};

struct [[nodiscard]] StrictNormalize {
    static constexpr bool operator() (Result status) {
        return status == Result::success;
    }
};

template <class F>
[[ [ : ^F : ] ] // expand into [ [nodiscard] ]
auto transform(auto... args) {
    return F()(args...);
};

int main() {
    transform<StrictNormalize>(Result::success); // warning on
    "nodiscard"
    bool isOk = transform<StrictNormalize>(Result::success); // OK
}
```

Here `transform` looks into the attributes attached to the callable `F`, and recovers the `[[nodiscard]]` attribute that originally appertained to `StrictNormalize` declaration.

We expect a number of applications for attribute introspection to happen in the context of code injection [P2237R0], where, for example, one may want to skip over

`[[deprecated]]` members. The following example demonstrates skipping over deprecated members:

```
struct User {
    [[deprecated]] std::string name;
    std::string uuidv5;
    [[deprecated]] std::string country;
    std::string countryIsoCode;
};

template<class T>
constexpr std::vector<std::meta::info> liveMembers(const T& user) {
    std::vector<std::meta::info> liveMembers;
    auto deprecatedAttribute = std::meta::attributes_of(^[[deprecated]])
        [0];
    auto keepLive = [&] <auto r> {
        if (!std::ranges::any_of(
            attributes_of(^T),
            [deprecatedAttribute] (auto meta) { meta == deprecatedAttributes;
        }) {
            liveMembers.push_back(r);
        }
    };

    template for (auto member : std::meta::members_of(^User)) {
        keepLive(member);
    }
    return os;
}

// Migrated user will no longer support deprecated fields
struct MigratedUser;
define_class(^MigratedUser, liveMembers(currentUser));
```

§ 2.1 Earlier work

While collecting feedback on this draft, we were redirected to [P1887R1], a pre existing proposal. In this paper, the author discusses two topics: ‘user defined attributes’ (also in [P2565R0]) and reflection over said attributes. We believe these two topics need not be conflated; both have intrinsic values on their own. We aim to focus this discussion entirely on **standard** attributes reflection. Furthermore the earlier paper has not seen work following the progression of [P2996R5], and so we feel this proposal is in a good place to fill that gap.

§ 2.2 Scope

Attributes are split into standard and non-standard. This proposal wishes to limit itself to standard attributes (9.12 [dcl.attr]). We feel that since it is up to the implementation to define how to handle non-standard attributes, it would lead to obscure situations that we

don't claim to tackle here.

A fairly simple (admittedly artificial) example can be built as such: Given an implementation supporting a non-standard `[[privacy::no_reflection]]` attributes that suppresses all reflection information appertaining to an entity, we would have a hard time coming up with a self-consistent system of rules to start with.

Henceforth, in this proposal, both 'attributes' and 'standard attributes' terms are meant to be equivalent terms. For interested readers we will re-open that topic in the [future direction](#) section.

§ 2.3 Optionality rule

There is a long standing and confusing discussion around the ignorability of attributes. We'll refer the reader to [\[P2552R3\]](#) for an at-length discussion of this problem, and especially with regard to what 'ignorability' really means. This proposal agrees with the discussion carried on there and in [\[CWG2538\]](#). We also feel that whether an implementation decides to semantically ignore a standard attribute should not matter.

Another interesting conversation takes place in [\[P3254R0\]](#), around the `[[no_unique_address]]` case, which serves again to illustrate that the tension around so called ignorability should not be considered a novel feature of this proposal.

What matters more is the following set of rules

- We should be able to statically reflect on appertaining attributes even if ignored: Implementations are required to parse and check syntactic rules related to attributes. Whether they are ignored semantically later on should not mean that they are removed from the source code during translation¹.
- Splicing attributes on a declaration should be undistinguishable from appertaining those attributes to this declaration.

§ 3 Proposed Features

We put ourselves in the context of [\[P2996R5\]](#) for this proposal to be more illustrative in terms of what is being proposed.

§ 3.1 `std::meta::info`

We propose that attributes be a supported *reflectable* property of the expression that is reflected upon. That means value of type `std::meta::info` should be able to represent an attribute in addition to the currently supported set.

§ 3.2 Reflection operator

The current proposition for reflection operator grammar does not cover attributes, i.e., the expression `^[deprecated]` is ill-formed. Our proposal advocates to support expression as following:

```
constexpr auto keepAttribute = ^[[nodiscard]];
```

The resulting value is a reflection value embedding relevant info for the *attribute* entity; in this case, the `nodiscard` attribute.

If the *attribute* is not a standard attribute, the expression is ill-formed.

§ 3.3 Splicers

We propose that the syntax

```
[[[: r :]]]
```

be supported in contexts where attributes are allowed.

- `[[[: r :]]]` produces a potentially empty *attribute-list* corresponding to the attributes reflected via reflection `r`.

Note that as it stands now *attribute-list* (9.12.1 [\[dcl.attr.grammar\]](#)) does not cover `alignas`. We understand that this limits potential use of the current proposal but also comes with difficulty, so this will be discussed in a separate paper.

A simple example of splicer using expansion statements is as follows. We create an augmented `enum` introducing a `begin` and `last` enumerator while preserving the original attributes:

```
[[nodiscard]]
enum class ErrorCode {
    warn,
    fatal,
};

[[[: ^ErrorCode :]]]
enum class ClosedErrorCode {
    begin,
    template for (constexpr auto e :
        std::meta::enumerators_of(^ErrorCode)) {
        return [:e:],
    }
    last,
};
```

If the attributes produced through introspection violate the rules of what attributes can appertain to what entity, the program is ill-formed, as usual.

§ 3.3.1 Attribute using

It is worth pointing out the interaction between *attribute-using-prefix* and splice expression that could lead to unexpected results, such as in the following example

```
auto unscopedAttribute = ^[[nodiscard]];
[[ using CC: debug, [: unscopedAttribute :] ]] enum class Code {};
```

While it is unlikely the user intends for the standard attributes to be targeted by `using CC`, current grammar says that the prefix applies to attributes as they are found in the *subsequent* list. To remediate this we can either enforce that *splice-name-qualifier* precedes *attribute-using-prefix* or have those constructs be mutually exclusive as they occur in `[[ ]]`.

To reduce the need to memorize unintuitive rules, we favor the later of those options, as following:

```
auto unscopedAttribute = ^[[nodiscard]];
[[ [: unscopedAttribute :] ]][[ using CC: debug ]] enum class Code {};
```

§ 3.4 Metafunctions

We propose to add two metafunctions to what has already been discussed in [P2996R5]. In addition, we will add support for attributes in the other metafunctions, when it makes sense.

§ 3.4.1 attributes_of

```
namespace std::meta {
    consteval auto attributes_of(info entity) -> vector<info>;
}
```

Applying this to a reflection entity will yield a sequence of `std::meta::info` representing each individual attribute appertaining to entity.

§ 3.4.2 is_attribute

```
namespace std::meta {
    consteval auto is_attribute(info entity) -> bool;
}
```

This would return true if the entity reflection designates a `[ [ attribute ] ]`; otherwise, it would return false.

§ 3.4.3 identifier_of, display_identifier_of

Given a reflection `r` designating a standard attribute, `identifier_of(r)` (resp. `u8identifier_of(r)`) is encouraged to return a `string_view` (resp. `u8string_view`) corresponding to the attribute-token.

A toy example follows

```
[[nodiscard, deprecated("Do not use me")]] int func();

void print_attributes(std::meta I) {
    template for (bool first{true}; constexpr auto e :
        std::meta::attributes_of(I)) {
        std::cout << (std::exchange(first, false) ? "" : ", ")
            << std::meta::identifier_of(e) << std::endl;
    }
}

print_attributes(^func); // Prints "nodiscard, deprecated"
```

Given a reflection `r` that designates an individual attribute, `display_identifier_of(r)` (resp. `u8display_identifier_of(r)`) returns an unspecified non-empty `string_view` (resp. `u8string_view`). Implementations are encouraged to produce text that is helpful in identifying the reflected attribute for display purpose. In the preceding example we could imagine printing `[[nodiscard]]` instead of `discard` as it is more suitable for display purpose to the user.

§ 3.4.4 `data_member_spec`, `define_class`

As it stands now, `define_class` allows piecewise building of a class via `data_member_spec`. However, to support attributes pertaining to those data members, we'll need to augment `data_member_options_t` to encode attributes we may want to attach to a data member.

The structure will change thusly:

```
namespace std::meta {
    struct data_member_options_t {
        struct name_type {
            template <typename T> requires
                constructible_from<u8string, T>
                consteval name_type(T &&);

            template <typename T> requires
                constructible_from<string, T>
                consteval name_type(T &&);
        };

        optional<name_type> name;
        bool is_static = false;
        optional<int> alignment;
    };
}
```

```

        optional<int> width;
-       bool no_unique_address = false;
+       vector<info> attributes;
    };
}

```

From there building a class piecewise proceeds as usual

```

struct Empty {};
struct [[nodiscard]] S;
define_class(^S, {
    data_member_spec(^int, {.name = "i"}),
    data_member_spec(^Empty, {.name = "e",
                             .attributes = {[no_unique_address]}})
});

// Equivalent to
// struct [[nodiscard]] S {
//     int i;
//     [[no_unique_address]] Empty e;
// };

```

Note here that [P2996R5] prescribes the use of `no_unique_address` and `alignment` as part of `data_member_options_t`. We think that this approach scale poorly in that every new attributes introduced into the standard will lead to discussions on whether or not they deserve to be included in `data_member_options_t`. There is also little explanations as to why those were picked among all. Leveraging reflected attributes through the approach we propose above we think is more in line with the philosophy of leveraging `std::meta::info` as a black box.

§ 3.4.5 Other metafunctions

For any reflection where `is_attribute` returns false, other metafunctions not listed above are not considered constant expressions

§ 3.5 Queries

We do not think it is necessary to introduce any additional query or queries at this point. We would especially not recommend introducing a dedicated query per attribute (e.g., `is_deprecated`, `is_nouniqueaddress`, etc.). Having said that, we feel those should be achievable via concepts, something akin to:

```

auto deprecatedAttributes = std::meta::attributes_of(^[[deprecated]]);

template<class T>
concept IsDeprecated = std::ranges::any_of(
    attributes_of(^T),

```



```

        [deprecatedAttributes] (auto meta) { meta ==
    deprecatedAttributes[0]; }
);

```

§ 4 Proposed wording

§ 4.1 Language

§ 4.1.1 [dcl.attr.grammar] Attribute syntax and semantics

Change the grammar to allow splicing attributes from reflection in 9.12.1 [dcl.attr.grammar] as follows

attribute-specifier:

```

[ [ attribute-using-prefixopt attribute-list ] ]
[ [ splice-name-qualifier ] ]

```

Following this subsection modify the paragraph 7

- 7 Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier* ~~or~~, within the *balanced-token-seq* of an *attribute-argument-clause* or as part of a *reflect-expression*.

Add the following paragraph

- 8 If an *attribute-specifier* contains a *splice-name-qualifier*, every standard attribute contained in that reflection are applied to the entity to which that *attribute-specifier* is attached. The form of this expansion shall be treated as an *attribute-list*. If *splice-name-qualifier* describes an entity that bear no attributes, it has no effect.

§ 4.1.2 [expr.unary.general] General

Augment the ^ operator to allow for reflection on attribute in 7.6.2.1 [expr.unary.general] as follows

reflect-expression:

```

^ ::
^ namespace-name
^ nested-name-specifieropt template-name
^ nested-name-specifieropt concept-name
^ type-id
^ id-expression
^ [ [ attribute ] ]

```

Note here that we are breaking the attributes appertaining rules for anything but [[assume]], (e.g) in the form ^[[nodiscard]]; the nodiscard attribute is now

appertaining to a null statement which is not allowed.

This is still an open discussion but a suggestion is as following where we individually address 9.12.4 [\[dcl.attr.depend\]](#).

Edit paragraph 1

- 1 The *attribute-token* carries_dependency specifies dependency propagation into and out of functions. No *attribute-argument-clause* shall be present. As part of a reflect-expression, the attribute must apply to the null statement. Outside of a reflect-expression the ~~The~~ attribute may be applied to a parameter of a function or lambda, in which case it specifies that the initialization of the parameter carries a dependency to ([intro.multithread]) each lvalue-to-rvalue conversion ([conv.lval]) of that object. The attribute may also be applied to a function or a lambda call operator, in which case it specifies that the return value, if any, carries a dependency to the evaluation of the function call expression.

§ 4.1.3 [\[expr.reflect\]](#) The reflection operator

Modify the subsection [\[expr.reflect\]](#) to describe reflection applying to attributes, adding the following paragraph

- 11 When applied to a `[ [ attribute ] ]`, the reflection operator produces a reflection for the indicated *attribute*. If *attribute* is not described in 9.12 [\[dcl.attr\]](#), the behavior is implementation defined.

§ 4.1.4 [\[expr.eq\]](#) Equality Operators

Augment the section in 7.6.10 [\[expr.eq\]](#) describing comparisons between `std::meta::info` to specify comparison on reflection over attributes

- Otherwise, if one operand represents a standard attribute, then they compare equal if and only if the other operand represents a standard attribute and both those attributes have identical *attribute-token*. If any or both of those operands have a *attribute-argument-clause*, the content of that clause is ignored.

§ 4.2 Library

§ 4.2.1 [\[meta.synop\]](#) Header `<meta>` synopsis

Add to the [\[meta.reflection.queries\]](#) section from the synopsis the two metafunctions as follows:

```
constexpr bool is_attribute(info r);
constexpr vector<info> attributes_of(info r);
```

§ 4.2.2 [\[meta.reflection.queries\]](#), Reflection queries

42 `constexpr bool is_attribute(info r);`

Returns: true if `r` designates a standard attribute. Otherwise, false.

43 `constexpr vector<info> attributes_of(info r);`

Returns: A vector containing a reflection for each of the attributes that appertain to the entity `r`

§ 5 Future direction

§ 5.1 Scoped attributes

The current proposal limits itself to attributes whose semantics are described by the standard, our argument was mostly based on leaving room to implementation to do what they want with the other kind of attributes. Also if the conversation around attributes automatically get redirected into a ‘ignorability’ side conversation, this is even more the case when it comes to non standard attributes... As we see that there is appetite for tagging various entities and apply dedicated logic based off those tags, this is something that we think we can accomodate by extending this proposal.

An example of extension would be to introduce

```
constexpr vector<info> attributes_of(info r, std::string_view ns);
```

Where we intend to filter the attributes returned from the `r` reflection, on a particular attribute-namespace. So in the following example

```
[[nodiscard, tag:Validation]] enum MyEnum { Success, Failure };
constexpr auto tags = std::meta::attributes_of(^MyEnum, "tag"); //
    Contains "tag:Validation"
```

Arguably this would reopen a heated discussion on how much implementations are allowed to ignore scoped attributes, and at what stage of parsing this is allowed. Based on this, and to avoid distraction we are not actively exploring this venue.

§ 5.2 Implementation

This proposal is being worked on in a fork from the Clang P2996 branch to be advertised at a later time. However and since more ambitious reflection related proposals have been implemented already, we do not foresee dramatic complexity.

§ 6 Conclusion

Originally the idea of introducing a `declattr(Expression)` keyword seemed the most straightforward approach to tackling this problem. However based on feedback, the concern of introspecting on expression attributes was a topic that belongs with the Reflection SG. The current proposal shifted away from the original `declattr` idea to align better with the reflection toolbox. Note also that, as we advocate here for `[[ [: r :] ]]` to be supported, we recover the ease of use that we first envisioned `declattr` to have.

§ 7 References

- [CWG2538] Jens Maurer. 2021-12-02. Can standard attributes be syntactically ignored?
<https://wg21.link/cwg2538>
- [P1887R1] Corentin Jabot. 2020-01-13. Reflection on attributes.
<https://wg21.link/p1887r1>
- [P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
<https://wg21.link/p2237r0>
- [P2552R3] Timur Doumler. 2023-06-14. On the ignorability of standard attributes.
<https://wg21.link/p2552r3>
- [P2565R0] Bret Brown. 2022-03-16. Supporting User-Defined Attributes.
<https://wg21.link/p2565r0>
- [P2996R5] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-08-14. Reflection for C++26.
<https://wg21.link/p2996r5>
- [P3254R0] Brian Bi. 2024-05-22. Reserve identifiers preceded by `@` for non-ignorable annotation tokens.
<https://wg21.link/p3254r0>
- [P3385R0] Aurelien Cassagnes, Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-09-16. Attributes reflection.
<https://wg21.link/p3385r0>

-
1. Discussion in <https://eel.is/c++draft/dcl.attr#depend-2>, imply that translation units need to carry their attributes unchanged to observe that rule. ↩